

模型選擇與 Token 成本控制

GPT-6 更新版：用適當的推理預算，完成可靠的軟體工程工作

Luna

高頻、低成本

Sol

日常工程主力

Astra

高難度判斷

版本：V2.0 | 查核日期：2026-09-27

適用：ChatGPT Codex／Codex Desktop／Codex CLI 的模型分流與 API 成本估算

摘要

在 Codex 中，最昂貴的錯誤通常不是「選錯模型」，而是把高能力模型與高推理強度設成長時間的常駐預設。當工作只是格式調整、局部修正、一般程式撰寫或可明確拆解的工程任務時，讓 Astra 以高推理強度反覆讀檔、呼叫工具與重跑測試，往往會投入超過必要的預算。

本更新版採用 GPT-6 家族的分層路由：Luna 處理範圍明確且高頻的工作；Sol 作為日常程式開發主力；Astra 處理跨檔案理解、架構決策與高風險審查。核心原則不變：先使用能達到品質門檻的最輕設定，只有在證據顯示品質不足時才升級。

本文清楚區分「OpenAI API 逐 Token 與工具計價」與「ChatGPT Codex 產品中的方案額度／速率限制」。前者可依公開費率估算；後者不可用 API 單價直接換算。

一句話結論：日常預設以 Sol Medium 為中心；小事降到 Luna Low；架構與審查升到 Astra Medium；只有難以定位或高風險的問題才升高 Astra 的推理強度。

1. 為什麼高推理設定很容易「一下就燒掉很多額度」？

推理模型在輸出最終答案之前，會使用內部 reasoning tokens 進行規劃、檢查替代方案、工具使用與錯誤修復。推理強度從 low、medium 提升到 high、xhigh 或 max，本質上是在允許模型投入更多推理工作。這可能提升困難任務的可靠度，但在簡單任務上也可能形成過度運算。

Codex 又不是單次問答工具，而是 Agent。一次「修好這個 bug」可能包含讀檔、搜尋、修改、執行測試、讀取錯誤、再次修改、再次驗證等多個循環。若每一輪都由高成本模型以高推理強度執行，使用量會快速累積。

此外，工作範圍也直接影響成本。掃描整個 repository、攜帶非常長的上下文、反覆載入大型檔案，以及要求「全部檢查、全部修正、全部重構」，都會顯著增加模型需要處理的資訊量。

2. 2026 年 GPT-6 家族的角色定位

模型	定位	適合工作	建議推理強度	使用原則
GPT-6 Luna	最高效率、低成本	小修改、明確範圍、批次例行工作	Low／Medium	能用 Luna 達標，就不要升級
GPT-6 Sol	日常工程主力	一般 coding、測試、API、腳本、常規除錯	Low／Medium	建議作為多數 Codex 任務的起點
GPT-6 Astra	旗艦級高難度工作	架構、跨模組、審查、深度推理	Medium 起步	只有量測到品質收益才升 High 以上

註：實際可見模型與推理選項會依 ChatGPT 方案、Codex 版本與工作區設定而不同；本表是工作分流建議，不是帳戶可用模型的保證。

3. 最新官方 API 計價方式（2026-09-27 查核）

以下為 OpenAI API 的標準處理文字費率，單位均為美元／每 100 萬 Token。短上下文與長上下文的分界及各處理層級，以官方 Pricing 頁當下顯示為準；長上下文會提高整筆請求的費率。

模型	短上下文輸入	快取輸入	快取寫入	輸出	長上下文輸入／輸出
gpt-6-luna	\$0.10	\$0.01	\$0.125	\$0.50	\$0.20 ／ \$0.75
gpt-6-sol	\$2.00	\$0.20	\$2.50	\$10.00	\$4.00 ／ \$15.00
gpt-6-astra	\$10.00	\$1.00	\$12.50	\$50.00	\$20.00 ／ \$75.00
gpt-5.3-codex	\$1.75	\$0.175	官方頁未列	\$14.00	以官方專用模型頁為準

最簡單的估算式：成本 ≈ （一般輸入 ÷ 1,000,000 × 輸入費率） + （快取輸入 ÷ 1,000,000 × 快取費率） + （輸出 ÷ 1,000,000 × 輸出費率） + 工具費。例如一次短上下文請求使用 10 萬一般輸入與 2 萬輸出 Token，gpt-6-sol 約為 \$0.40；gpt-6-luna 約為 \$0.02；gpt-6-astra 約為 \$2.00，均未含工具費與快取寫入。

工具也會計價：官方目前列示 Web Search 為 \$10／1,000 次呼叫，另加檢索內容 Token 的模型費；Hosted Shell／Code Interpreter 依容器規格與工作階段計費。Fast、Batch、Flex 與區域資料駐留並非標準費率，使用前須回到官方 Pricing 頁核對。

4. 工作類型 × 模型 × 推理強度建議表

工作類型	推薦模型	推理強度	相對耗用	建議
README、註解、命名、格式整理	Luna	Low	★	首選
批次改檔名、路徑、重複設定	Luna	Low	★	首選
建立簡單骨架／模板	Luna／Sol	Low	★-★★	很適合
HTML／CSS 小修改	Sol	Low	★★	日常
一般 Python／JS／Arduino	Sol	Low-Medium	★★-★★★★	日常主力
ESP32 感測器／Gateway	Sol	Medium	★★★★	主力
API、CRUD、資料處理	Sol	Medium	★★★★	主力
Unit／Integration Test	Sol	Medium	★★★★	主力
一般 Bug 修復	Sol	Medium	★★★★	主力
跨多檔案重構	Sol／Astra	Medium	★★★★-★★★★★	視範圍升級
新功能跨模組	Astra	Medium	★★★★★	重要工作
系統架構／Agent／MCP 設計	Astra	Medium	★★★★★	先規劃後下放實作
Code Review／架構審查	Astra	Medium-High	★★★★★	品質關卡

工作類型	推薦模型	推理強度	相對耗用	建議
難重現 Bug／並行問題	Astra	High	★★★★★	救火
大型遷移／安全性／關鍵決策	Astra	High 以上	★★★★★	最後手段

5. 最實用的「四檔模式」

第 1 檔 | Luna Low

雜工模式：文字、格式、命名、重複性修改、清楚的小任務。目標是快、便宜、做完即停。

第 2 檔 | Sol Low／Medium

日常工程模式：一般程式撰寫、ESP32、Python Gateway、API、測試與常規除錯。建議成為多數 Codex 任務的預設。

第 3 檔 | Astra Medium

專業判斷模式：跨檔案理解、架構規劃、重要 refactor、review、完整性要求較高的工程任務。

第 4 檔 | Astra High 以上

專家會診模式：多次失敗仍無法定位、複雜 concurrency、重大遷移、安全性或高風險決策。問題解完後立刻降檔。

6. 比「選模型」更重要的 6 個省用量策略

策略 1：限制工作範圍。不要說「檢查整個專案」。改成「只檢查 /gateway；最多找 3 個直接相關問題；不要修改其他資料夾」。

策略 2：Plan 與 Execute 分開。先讓 Astra Medium 只做 root cause 與修復計畫；確認方向後，把大量實作交給 Sol Medium。避免高階模型從頭做到尾。

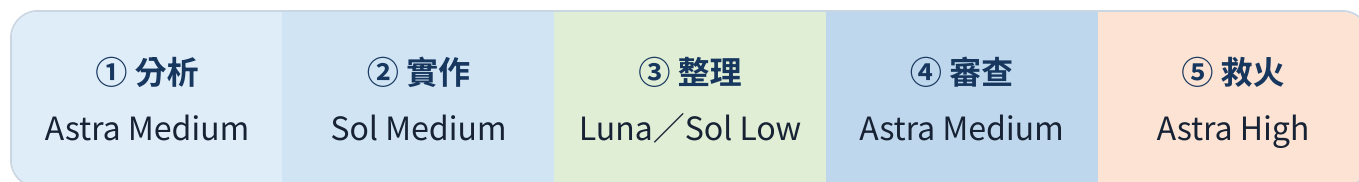
策略 3：採用 Minimum Necessary Change。明確要求不要做無關 refactor、不要擴大範圍、修改後只執行相關測試。

策略 4：測試通過即停止。Agent 容易在完成後繼續「順便改善」。加上 stop condition：相關測試通過且需求滿足後停止，不再延伸。

策略 5：高階模型只做 Quality Gate。架構規劃、review、疑難診斷使用 Astra；大量 coding、修檔、測試交給 Sol/Luna。

策略 6：難題結束立即降檔。高推理設定很容易被忘在模型選擇器中。解完難題後，下一個任務前先確認是否已切回日常檔位。

7. 推薦的 Codex 工作流：高階模型做腦，大量實作交給均衡模型



這個流程的重點不是追求「最強模型從頭包到尾」，而是把昂貴的推理集中在最需要判斷的節點。真正耗時但規格明確的工作，通常可以交給更便宜的模型完成。

8. 實例：ESP32 → Python Gateway → Supabase → Dashboard

階段	推薦模型	理由
Architecture／資料流	Astra Medium	先確認系統邊界、介面與責任分工
Repo structure	Sol Low	規格清楚、低風險
ESP32 firmware	Sol Medium	需要 coding 與基本除錯
Python Gateway	Sol Medium	一般工程實作
Supabase schema／CRUD	Sol Medium	規則明確但需完整性
測試與 CI	Sol Medium	適合大量可驗證工作
README／操作文件	Luna／Sol Low	高頻低風險
Architecture review	Astra Medium	需要跨模組整體判斷
三次以上仍解不掉的 bug	Astra High	此時更多推理才值得

9. 可直接貼進 Codex 的「節省用量工作規則」

工作原則：

1. 優先採用 minimum necessary change。
2. 不要掃描整個 repository，除非目前證據不足。
3. 先讀取與問題直接相關的檔案。
4. 不做與任務無關的 refactor。
5. 先定位 root cause，再進行修改。
6. 修改後只執行與本次變更直接相關的測試。
7. 若測試通過且需求已滿足，立即停止，不延伸改善。
8. 若需要擴大工作範圍，先說明原因與預計新增讀取的區域。

10. 決策樹：每次工作前只問 4 個問題

Q1：任務是否範圍明確、答案近乎機械式？

是 → Luna Low；否 → Q2

Q2：主要工作是不是一般 coding、測試、API、腳本或常規除錯？

是 → Sol Low/Medium；否 → Q3

Q3：是否需要跨檔案理解、架構判斷、審查或高完整度？

是 → Astra Medium；否 → 回 Sol Medium

Q4：Astra Medium 已失敗、問題高度複雜或風險極高嗎？

是 → Astra High/更高推理；完成後立即降檔

11. 關於 API Token 價格與 Codex 額度：不要混為一談

OpenAI API 會公布模型每百萬輸入／輸出 Token 的價格與工具費，因此可以做出可驗算的 API 成本預估。此頁的 gpt-5.3-codex 價格也只適用於 API 呼叫；它不等於桌面版 Codex 工作階段的扣點規則。

ChatGPT Codex 的方案額度、credits、速率限制與產品內部計量方式，會依方案、模型、推理強度、上下文、Agent 多步循環與工具呼叫而變動。若在介面看到「額度很快下降」，不能把它簡化成「可見輸出 Token × API 單價」。最可靠的優化仍是縮小任務範圍、降低不必要的推理強度、避免無關掃描，並建立模型分流制度。

12. 建議配置（實務版）

情境	建議
不知道該選什麼	先用 Sol Medium
任務非常明確	降到 Sol Low 或 Luna Low
只有整理／改名／格式	Luna Low
要規劃架構	Astra Medium
要做重要 Code Review	Astra Medium，必要時 High
連續 2-3 次修復仍失敗	升 Astra High
高階模型已解決問題	立刻切回 Sol Medium

如果希望更制度化，可以把這套規則直接寫入專案 AGENTS.md 或開發規範中，讓「什麼任務用什麼模型」成為工程流程的一部分，而不是每次臨時憑感覺選擇。

結論

節省 Codex 用量的核心，不是永遠選最便宜的模型，而是建立「任務難度、模型能力、推理強度」三者之間的匹配。高階模型的價值在於處理高價值判斷，而不是承包所有低風險工作。

Luna 做小事 | Sol 做日常工程 | Astra 做判斷與審查 | Astra High 負責救火

官方資料來源

- [OpenAI API Pricing](#)：GPT-6 Astra／Sol／Luna、長上下文、快取、工具與處理層級費率；本文所有金額均於 2026-09-27 依此頁查核。
- [OpenAI Models](#)：最新模型角色與可用能力。
- [OpenAI Model Guidance](#)：GPT-6 的模型選擇與推理使用建議。